UT2
Programación multihilo

Programación de servicios y procesos

2º Desarrollo de Aplicaciones Multiplataforma

RA2. Desarrolla aplicaciones compuestas por varios hilos de ejecución analizando y aplicando librerías específicas del lenguaje de programación – 35%

a) Se han identificado situaciones en las que resulte útil la utilización de varios hilos en un programa.
b) Se han reconocido los mecanismos para crear, iniciar y finalizar hilos.
c) Se han programado aplicaciones que implementen varios hilos.
d) Se han identificado los posibles estados de ejecución de un hilo y programado aplicaciones que los gestionen.
e) Se han utilizado mecanismos para compartir información entre varios hilos de un mismo proceso.
f) Se han desarrollado programas formados por varios hilos sincronizados mediante técnicas específicas.
g) Se ha establecido y controlado la prioridad de cada uno de los hilos de ejecución.
h) Se han depurado y documentado los programas desarrollados.
i) Se ha analizado el contexto de ejecución de los hilos
j) Se han analizado librerías específicas del lenguaje de programación que permiten la programación multihilo.
k) Se han reconocido los problemas derivados de la compartición de información entre los hilos de un mismo proceso.

EVALUACIÓN

- Actividades en el aula/aula virtual ---------- 10%
- Entrega de trabajos/prácticas ---------- 20%
- Prueba teórica o práctica ---------- 70%

¿Qué vamos a trabajar?

- Identificar situaciones en las que puede ser ventajoso desarrollar una aplicación multihilo.
- Conocer las técnicas básicas de programación para desarrollar aplicaciones multihilo.
- Crear y lanzar desde el hilo inicial de un programa varios hilos que compartan información.
- Asegurar la exclusión mutua en el acceso a la información compartida por varios hilos.
- Utilizar técnicas específicas para evitar el interbloqueo.
- Aprender los mecanismos que proporciona Java para esperar a que un hilo termine su ejecución, para interrumpir su ejecución y para establecer su prioridad.
- Identificar los potenciales problemas de la programación multihilo y las técnicas existentes para evitarlos.
- Diferenciar los distintos paquetes y clases existentes en la programación multihilo y asíncrona.

¿Qué vamos a trabajar?



Contenidos de la unidad

1 Hilos y procesos
Qué es un hilo · procesos frente a hilos · secuencial frente a hilos

2 Crear y gestionar hilos
Thread y Runnable · estados · join · interrupciones · prioridad

3 Compartir información
Recursos compartidos · condiciones de carrera · atomicidad · visibilidad

4 Sincronización
synchronized · métodos y bloques · variables atómicas

5 Interbloqueo e inanición
Problemas de vivacidad · cómo detectarlos y evitarlos

6 Coordinación de hilos
Productor-consumidor · wait(), notify() y notifyAll()

7 java.util.concurrent
Semáforos · ExecutorService · Callable y Future · colecciones concurrentes

8 Depuración y documentación
Depurador · volcados de hilos · VisualVM · buenas prácticas

Bloque 1
Hilos y procesos

UT2. Programación multihilo

Qué es un hilo · procesos frente a hilos · secuencial frente a hilos

¿Qué es un hilo?

- **Un Thread (o hilo o subproceso) es**
  - La unidad básica de ejecución de un SO
  - Su característica es que permite a una aplicación realizar varias tareas a la vez
  - Las partes que corren separadamente dentro de un programa
  - La secuencia de código que está en ejecución dentro del contexto de un proceso

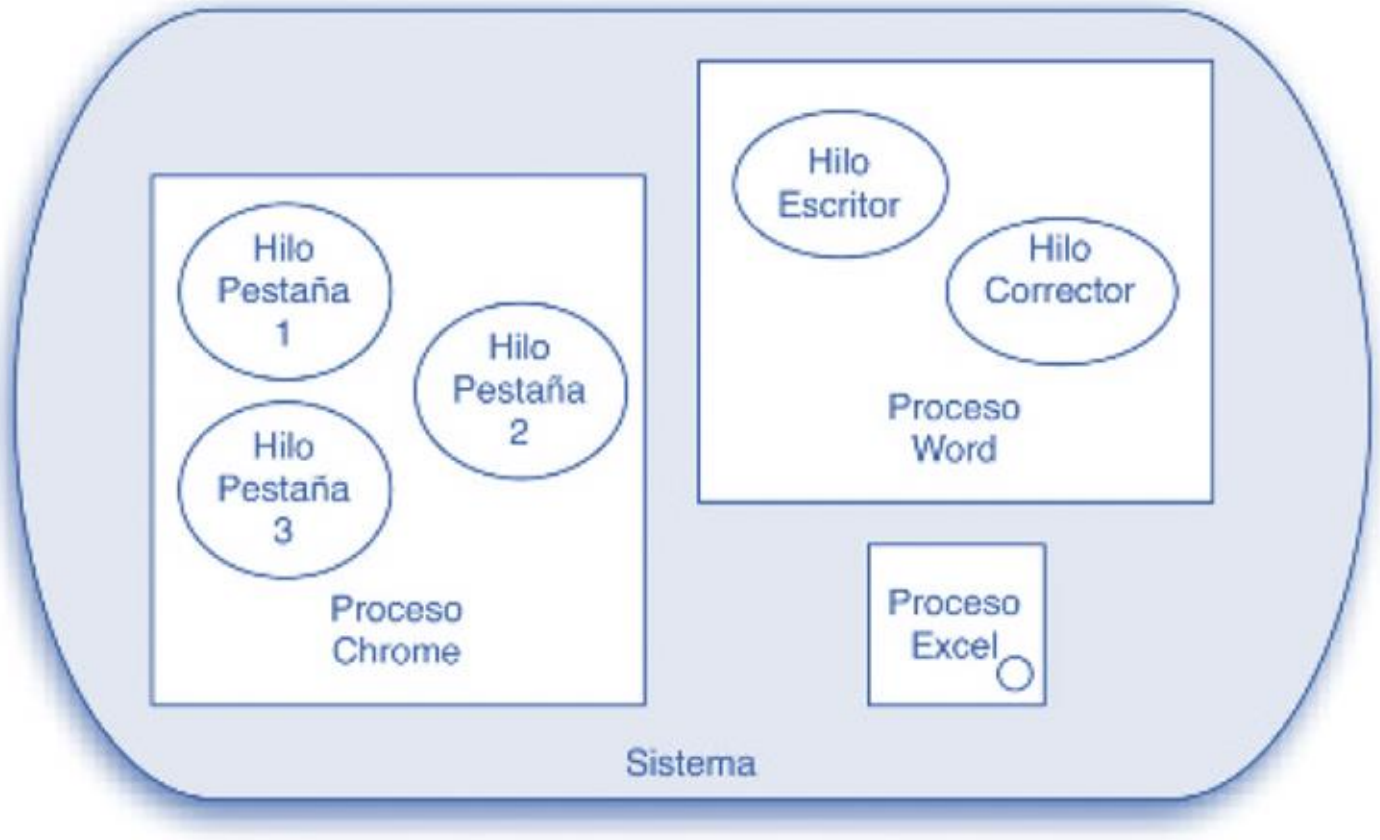
- **En Java, al lanzar un proceso se crea un Thread que ejecuta el main.**

Procesos vs hilos.

- Al crear un proceso, el SO le asigna recursos y memoria que necesita para ejecutarse (y que conforman su contexto)
- Los hilos se ejecutan dentro del contexto de un programa.
- Los procesos tienen espacios de memoria independientes.
- Los hilos que dependen de un mismo proceso, comparten la memoria del proceso entre ellos.
- Un programa en ejecución (proceso) puede realizar distintas tareas (threads) al mismo tiempo.

Procesos vs hilos.

- Ejemplos.



Procesos vs hilos: ventajas y desventajas

- **Los programas pueden delegar tareas a hilos sin perder la capacidad de atender a las peticiones del usuario.**
  - Sin hilos, si el usuario pide una tarea larga a un programa, este aparecerá bloqueado hasta que la tarea se haya completado
  - Con hilos el programa principal seguirá respondiendo a los comandos del usuario mientras la tarea se ejecuta en un hilo distinto.

- **Los hilos comparten entre ellos la memoria y los recursos asociados al proceso al que pertenecen.**
  - Pueden comunicarse entre ellos usando la memoria compartida.
  - Al compartir memoria, la sincronización se vuelve más compleja

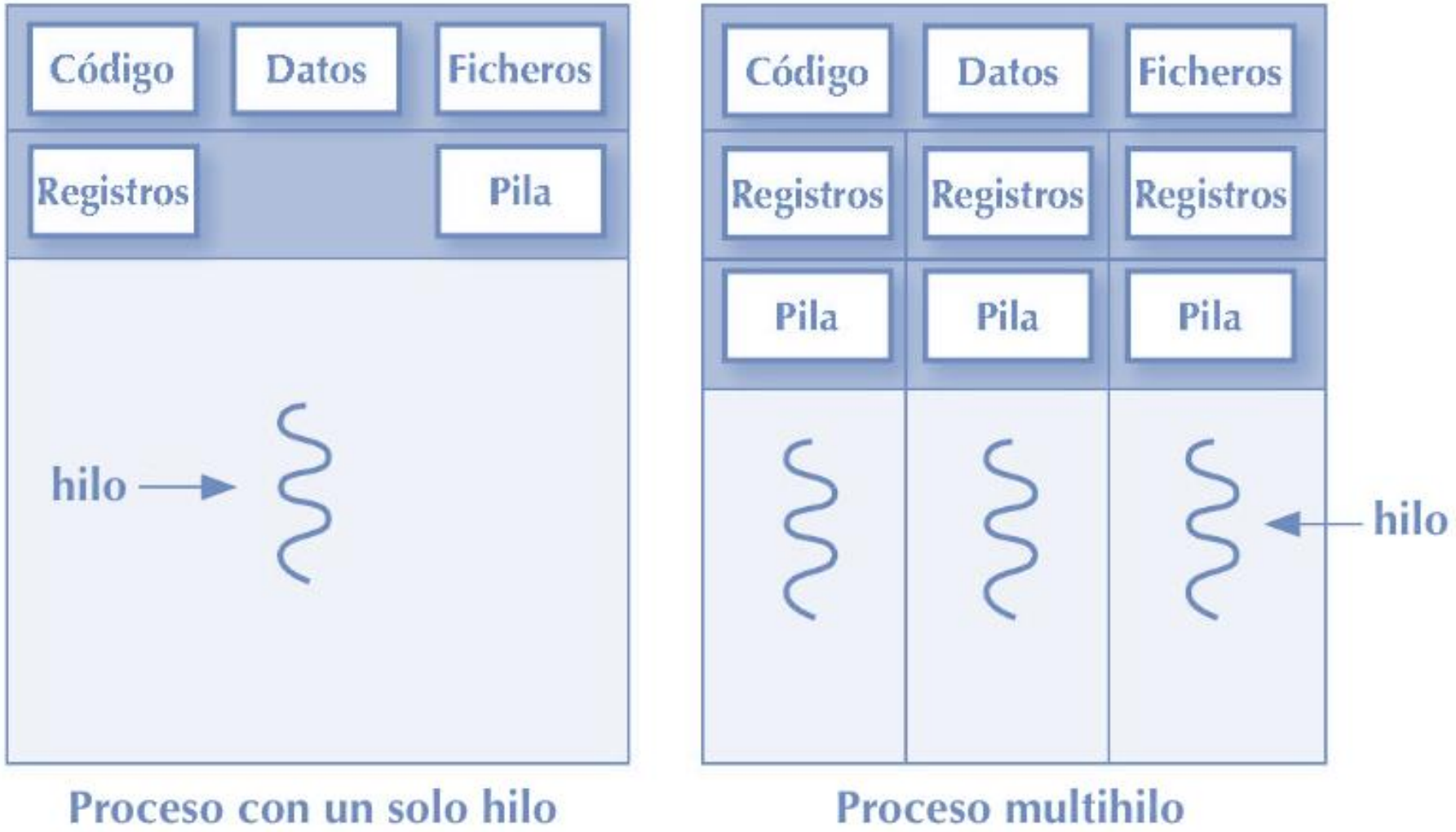
Procesos vs hilos: ventajas y desventajas

- **Crear hilos consume mucha menos memoria que crear procesos: comparten la memoria del proceso y solo necesitan su propia pila y registros**

- **La presencia de hilos permite el uso real de arquitecturas con varios núcleos.**
  - Cuando un proceso multihilo está en ejecución, se pueden aprovechar de forma paralela los varios núcleos del procesador.

- **El cambio de contexto entre hilos es más rápido que el cambio de contexto entre procesos**

Recursos compartidos por hilos.



Ejecución secuencial frente a hilos

Ejecución secuencial – Raton.java

```
public class Raton {
   private String nombre;
   private int tiempoAlimentacion;

   public static void main(String[] args) {
      long inicio = System.currentTimeMillis();
      Raton tom = new Raton("Tom", 4);
      Raton jerry = new Raton("Jerry", 5);
      Raton mickey = new Raton("Mickey", 3);
      Raton cerebro = new Raton("Cerebro", 6);

      tom.comer();
      jerry.comer();
      mickey.comer();
      cerebro.comer();
      System.out.println("Tiempo total: "
            + (System.currentTimeMillis() - inicio) / 1000 + " s");
   }

   // constructor: guarda nombre y tiempoAlimentacion

   public void comer() {
      try {
         System.out.println("El ratón " + nombre + " ha comenzado a alimentarse");
         Thread.sleep(tiempoAlimentacion * 1000);
         System.out.println("El ratón " + nombre + " ha terminado.");
      } catch (InterruptedException e) {
         e.printStackTrace();
      }
   }
}
```

- Cada `comer()` empieza cuando **termina el anterior**.
- Un único hilo, el del `main`, hace todo el trabajo.
- Tiempo total: 4 + 5 + 3 + 6 = **18 s**.

```
El ratón Tom ha comenzado a alimentarse
El ratón Tom ha terminado.
El ratón Jerry ha comenzado a alimentarse
El ratón Jerry ha terminado.
El ratón Mickey ha comenzado a alimentarse
El ratón Mickey ha terminado.
El ratón Cerebro ha comenzado a alimentarse
El ratón Cerebro ha terminado.
Tiempo total: 18 s
```

Ejecución secuencial frente a hilos

Ejecución con hilos – RatonHilos.java

```
public class RatonHilos extends Thread {
   private String nombre;
   private int tiempoAlimentacion;

   public static void main(String[] args) {
      RatonHilos tom = new RatonHilos("Tom", 4);
      RatonHilos jerry = new RatonHilos("Jerry", 5);
      RatonHilos mickey = new RatonHilos("Mickey", 3);
      RatonHilos cerebro = new RatonHilos("Cerebro", 6);

      tom.start();
      jerry.start();
      mickey.start();
      cerebro.start();

      System.out.println("El main ha lanzado a los ratones y termina");
   }

   // constructor: igual que en Raton

   @Override
   public void run() {
      this.comer();
   }

   // comer(): igual que en Raton
}
```

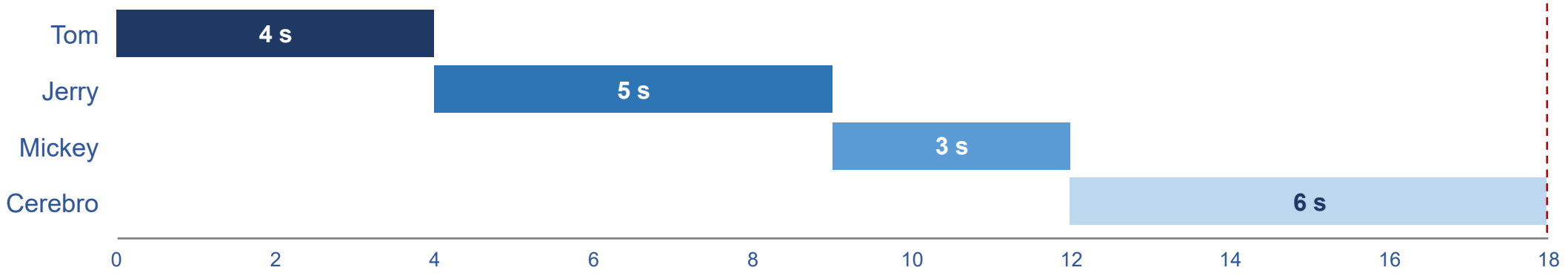
- Cada ratón es un hilo: `extends Thread`.
- `start()` crea el hilo y vuelve enseguida; el hilo ejecuta `run()`, que llama a `comer()`.
- Los 4 comen **a la vez**: el programa acaba con el más lento, **6 s**.
- El `main` termina el primero.

```
El main ha lanzado a los ratones y termina
El ratón Tom ha comenzado a alimentarse
El ratón Jerry ha comenzado a alimentarse
El ratón Cerebro ha comenzado a alimentarse
El ratón Mickey ha comenzado a alimentarse
El ratón Mickey ha terminado.
El ratón Tom ha terminado.
El ratón Jerry ha terminado.
El ratón Cerebro ha terminado.
```

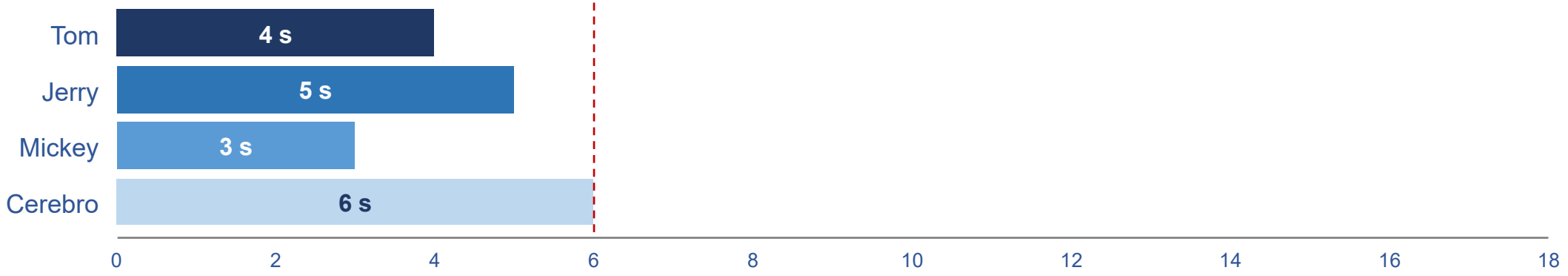
Ejecución secuencial frente a hilos

Comparación en el tiempo

Secuencial (Raton): 18 s



Con hilos (RatonHilos): 6 s



Cambios en el código: `extends Thread` · `run()` llama a `comer()` · `start()` en lugar de `comer()`.

Bloque 2
Crear y gestionar hilos

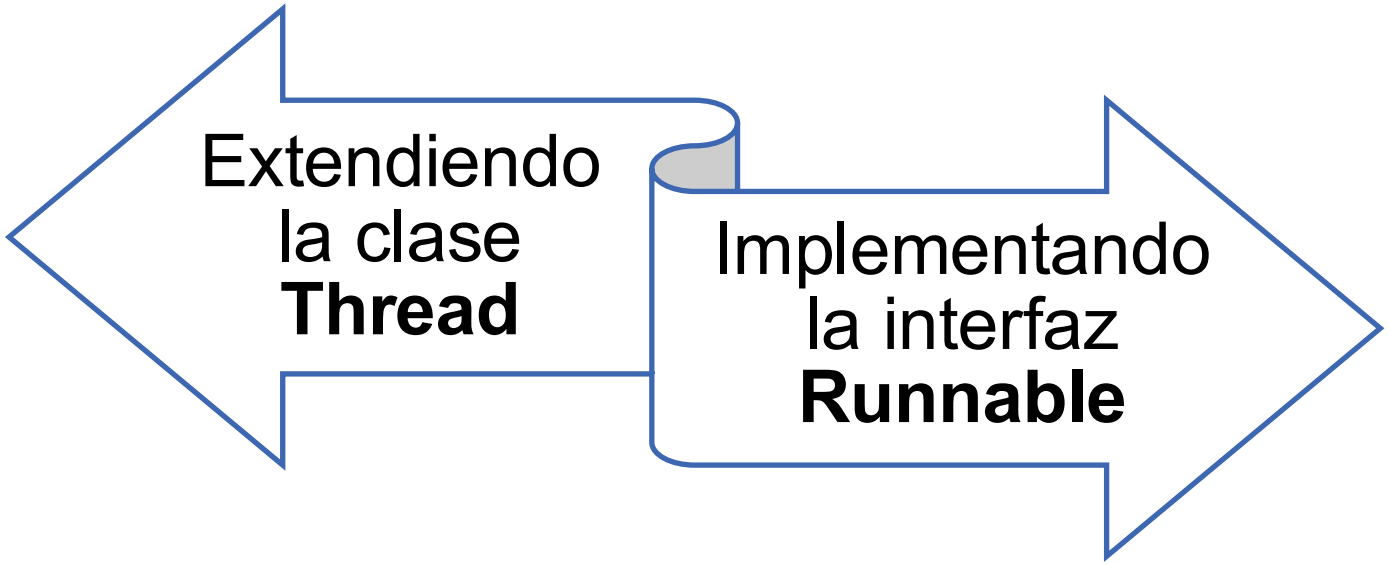
UT2. Programación multihilo

Thread y Runnable · estados · join · interrupciones · prioridad

Crear un hilo

- Crear un hilo es más eficiente que crear un proceso

- Hay dos formas de crear un Thread:



Implementar la interfaz Runnable

Paso a paso

1. La clase implementa la interfaz `Runnable`.
2. Escribe en `run()` el trabajo del hilo. Es como el `main` del hilo: cuando `run()` acaba, el hilo termina.
3. Crea un `Thread` y pásale el objeto.
4. Llama a `start()`: se crea el hilo nuevo y **él** ejecuta `run()`.

```java
public class EjemploRunnable implements Runnable {
    @Override
    public void run() {
        System.out.println("Soy EjemploRunnable");
    }
}
```

Cómo se lanza (en el main):

```java
Thread t = new Thread(new EjemploRunnable());
t.start();
```

¿Por qué no llamar a run() directamente?

Porque `run()` es un método normal: se ejecutaría en el hilo que lo llama y no se crearía ningún hilo (lo veremos con los ratones).

Salida:

```
Soy EjemploRunnable
```

Extender la clase Thread

Paso a paso

1. La clase hereda de `Thread` (que ya implementa `Runnable`).
2. Sobrescribe `run()` con el trabajo del hilo.
3. Crea el objeto: **ya es un hilo**, no hace falta otro `Thread`.
4. Llama a su `start()`.

```
public class EjemploThread extends Thread {
    @Override
    public void run() {
        System.out.println("Soy EjemploThread");
    }
}
```

Cómo se lanza (en el main):

```
EjemploThread t = new EjemploThread();
t.start();
```

Lo que hereda de Thread

Dentro de `run()` puede usar directamente `getName()`, `isInterrupted()`, `sleep()`… A cambio, la clase ya no puede heredar de ninguna otra.

Salida:

```
Soy EjemploThread
```

Lanzar los dos tipos de hilo

PruebaRyT.java – ¿en qué orden salen los mensajes?

```
public class PruebaRyT {
   public static void main(String[] args) {
      System.out.println("Inicia el proceso.");
      new EjemploThread().start();
      Thread t = new Thread(new EjemploRunnable());
      t.start();
      System.out.println("Termina el proceso.");
   }
}
```

Ejecución 1

```
Inicia el proceso.
Soy EjemploThread
Termina el proceso.
Soy EjemploRunnable
```

Ejecución 2

```
Inicia el proceso.
Soy EjemploThread
Soy EjemploRunnable
Termina el proceso.
```

- El `main` también es un hilo: sigue ejecutándose mientras los otros dos arrancan.
- El orden **cambia entre ejecuciones**: lo decide el planificador del sistema operativo, no el orden del código.
- «Termina el proceso.» puede salir **antes** que los hilos: `main` no los espera (lo resolveremos con `join()`).
- El programa no acaba cuando acaba `main`, sino cuando acaban **todos** sus hilos.

Implementar la interfaz Runnable

Los ratones con Runnable – RatonRunnable.java

- El ratón ya no es un hilo: es **la tarea** (`implements Runnable`).
- Para ejecutarla se crea un `Thread` pasándole el ratón y se llama a `start()`.
- El resultado es el mismo que con RatonHilos: los 4 comen a la vez y todo acaba en **6 s**.
- Ventaja: RatonRunnable podría heredar de otra clase.

```
public class RatonRunnable implements Runnable {
   private String nombre;
   private int tiempoAlimentacion;

   public static void main(String[] args) {
      RatonRunnable tom = new RatonRunnable("Tom", 4);
      RatonRunnable jerry = new RatonRunnable("Jerry", 5);
      RatonRunnable mickey = new RatonRunnable("Mickey", 3);
      RatonRunnable cerebro = new RatonRunnable("Cerebro", 6);

      new Thread(tom).start();
      new Thread(jerry).start();
      new Thread(mickey).start();
      new Thread(cerebro).start();
   }

   // constructor y comer(): igual que en Raton

   @Override
   public void run() {
      this.comer();
   }
}
```

¿Thread o Runnable?

El hilo y la tarea

extends Thread

RatonHilos extends Thread

- Es **el hilo y la tarea a la vez**.
- Hereda `start()`, `join()`, `getName()`… y sobrescribe `run()` → `comer()`.

```
RatonHilos tom = new RatonHilos("Tom", 4);
tom.start();
```

implements Runnable

RatonRunnable implements Runnable

- Es solo **la tarea**: el método `run()` → `comer()`.

se la pasamos a

new Thread(tom)

- **El hilo** (el trabajador) que ejecuta la tarea.

```
RatonRunnable tom = new RatonRunnable("Tom", 4);
new Thread(tom).start();
```

Un `Thread` siempre ejecuta un `run()`. Con **extends Thread** el `run()` está en el propio hilo; con **implements Runnable** escribimos la tarea aparte y se la damos a un `Thread`.

Analogía: el **Thread es el cocinero** y el **Runnable es la receta**. Con extends Thread cada cocinero nace sabiendo una sola receta; con Runnable cualquier cocinero puede preparar cualquier receta.

¿Thread o Runnable?

Diferencias en la práctica

	extends Thread	implements Runnable
¿Qué es la clase?	Un hilo	Una tarea que ejecutará un hilo
Cómo se lanza	`tom.start();`	`new Thread(tom).start();`
Herencia	Ya no puede heredar de otra clase	Puede heredar de otra clase
Nombre del hilo	`getName()`	`Thread.currentThread().getName()`
Compartir datos	Cada hilo es un objeto distinto con sus atributos	Varios hilos pueden ejecutar el mismo objeto y compartir sus atributos
Pools de hilos (bloque 7)	No tiene sentido: ya es un hilo	`pool.submit(tarea)` acepta un Runnable

Regla práctica

Implementa **Runnable** salvo en ejemplos muy cortos: separa la tarea del hilo, deja libre la herencia y es lo que usan los pools de hilos. El resultado al ejecutar es el mismo.

¿Thread o Runnable?

start() frente a run(): el error más típico

- `start()` **crea un hilo nuevo**, y ese hilo llama a `run()`.
- Si llamamos a `run()` directamente **no se crea ningún hilo**: el `main` lo ejecuta como un método normal.
- Los ratones vuelven a comer uno detrás de otro: 2 + 3 = 5 s en lugar de 3 s.
- Pasa igual con extends Thread y con Runnable. Además, un hilo solo se puede arrancar **una vez**.

```
--- Llamando a run() ---
El ratón Tom come en el hilo main
El ratón Jerry come en el hilo main
Tiempo: 5 s
--- Llamando a start() ---
El ratón Tom come en el hilo Tom
El ratón Jerry come en el hilo Jerry
Tiempo: 3 s
```

```
    public static void main(String[] args) throws InterruptedException {
        System.out.println("--- Llamando a run() ---");
        long inicio = System.currentTimeMillis();
        RatonRunVsStart tom = new RatonRunVsStart("Tom", 2);
        RatonRunVsStart jerry = new RatonRunVsStart("Jerry", 3);
        tom.run();       // ¡error! no se crea ningún hilo
        jerry.run();
        long segundos = (System.currentTimeMillis() - inicio) / 1000;
        System.out.println("Tiempo: " + segundos + " s");

        System.out.println("--- Llamando a start() ---");
        inicio = System.currentTimeMillis();
        // objetos nuevos: un hilo solo se puede arrancar una vez
        tom = new RatonRunVsStart("Tom", 2);
        jerry = new RatonRunVsStart("Jerry", 3);
        tom.start();     // crea un hilo nuevo, que ejecuta run()
        jerry.start();
        tom.join();
        jerry.join();
        segundos = (System.currentTimeMillis() - inicio) / 1000;
        System.out.println("Tiempo: " + segundos + " s");
    }
}
```

Estados de un hilo.

Durante el ciclo de vida de los hilos, estos pasan por diversos estados. En Java, están recogidos dentro de la enumeración *State* contenida dentro de la clase *Thread.*

El estado de un hilo se obtiene mediante el método *getState()* de la clase *Thread*.

Estado	Valor en Thread.State	Descripción
Nuevo	NEW	El hilo está creado, pero aún no se ha arrancado.
Ejecutable	RUNNABLE	El hilo está arrancado y podría estar en ejecución o pendiente de ejecución
Bloqueado	BLOCKED	Esperando para adquirir un monitor ocupado por otro hilo
Esperando	WAITING	El hilo está esperando a que otro hilo realice una acción determinada
Esperando un tiempo	TIMED_WAITING	El hilo está esperando a que otro hilo realice una acción determinada en un período de tiempo concreto
Finalizado	TERMINATED	El hilo ha terminado su ejecución.

Estados de un hilo.

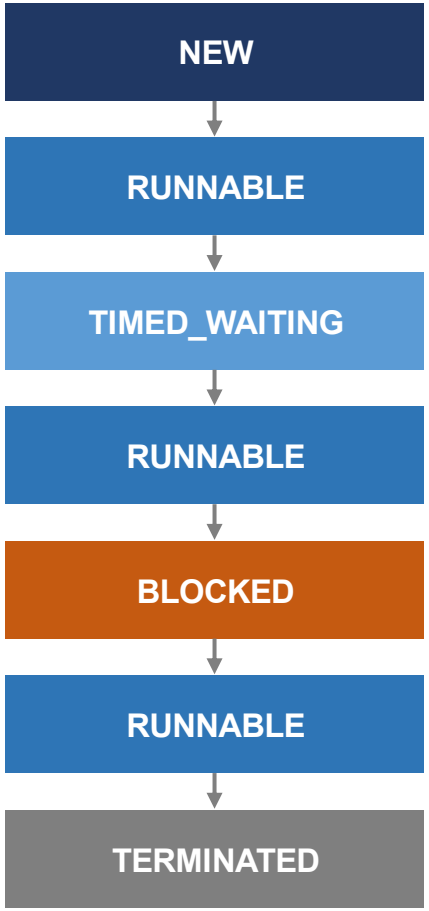
Ejemplo (1/2): ¿por qué estados pasa el ratón?

```
public class RatonEstados implements Runnable {
   static final Object comedero = new Object();

   private final String nombre;
   private final int tiempoAlimentacion;

   public RatonEstados(String nombre, int tiempoAlimentacion) {
      this.nombre = nombre;
      this.tiempoAlimentacion = tiempoAlimentacion;
   }

   @Override
   public void run() {
      try {
         System.out.println("El ratón " + nombre + " empieza a comer");
         Thread.sleep(tiempoAlimentacion * 1000L);   // TIMED_WAITING
         synchronized (comedero) {                    // BLOCKED si está ocupado
            System.out.println("El ratón " + nombre + " bebe agua");
         }
         System.out.println("El ratón " + nombre + " ha terminado");
      } catch (InterruptedException e) {
         Thread.currentThread().interrupt();
      }
   }
}
```



`new Thread(mickey)`: creado, aún sin arrancar

`start()`: el ratón empieza a comer

`Thread.sleep(2000)`: come durante 2 s

se despierta y va hacia el comedero

`synchronized (comedero)`: lo tiene otro hilo, espera

consigue el comedero y bebe

`run()` termina

Cada línea de `run()` lleva al hilo a un estado distinto.

Estados de un hilo.

Ejemplo (2/2): observar los estados desde el main – EstadosHilo.java

```
public static void main(String[] args) throws InterruptedException {
   RatonEstados mickey = new RatonEstados("Mickey", 2);
   Thread m = new Thread(mickey);

   // Otro hilo coge el comedero durante 3 s
   Ocupante ocupante = new Ocupante();
   ocupante.start();
   Thread.sleep(100);                    // nos aseguramos de que ya lo tiene

   ArrayList<Thread.State> estados = new ArrayList<>();
   estados.add(m.getState());            // NEW: creado, sin arrancar
   m.start();

   // Consultamos el estado sin parar y apuntamos cada cambio
   Thread.State anterior = m.getState();
   estados.add(anterior);
   while (anterior != Thread.State.TERMINATED) {
      Thread.State actual = m.getState();
      if (actual != anterior) {
         estados.add(actual);
         anterior = actual;
      }
   }

   StringBuilder sb = new StringBuilder();
   for (int i = 0; i < estados.size(); i++) {
      if (i > 0) sb.append(" → ");
      sb.append(estados.get(i));
   }
   System.out.println("Estados de Mickey: " + sb);
}
```

Salida:

```
Estados de Mickey: NEW → RUNNABLE →
TIMED_WAITING → RUNNABLE → BLOCKED →
RUNNABLE → TERMINATED
```

- **Ocupante** es otro hilo que coge el comedero 3 s: cuando el ratón acaba de comer (2 s), tiene que esperar → BLOCKED.
- El `main` consulta `getState()` sin parar y apunta cada **cambio** de estado.
- **WAITING** no aparece: es el de un hilo que espera sin límite de tiempo (`join()`, `wait()`). Lo veremos más adelante.

Métodos principales de la clase *Thread*

Método	Descripción
start	Arranca el hilo: la JVM crea un nuevo hilo que ejecuta run().
run	Método en el que se programa el bloque asíncrono. Se ejecuta cuando se invoca el método start.
join	Bloquea el hilo actual hasta que termina el hilo referenciado.
sleep	Método estático que detiene temporalmente la ejecución del hilo.
getId	Devuelve el identificador (long) del hilo. Desde Java 19: threadId().
getName	Devuelve el nombre del hilo.
getState	Devuelve el estado del hilo.
interrupt	Solicita la interrupción del hilo (activa su indicador).
interrupted	Método estático que comprueba si el hilo actual ha sido interrumpido y limpia el indicador.
isInterrupted	Comprueba si ese hilo ha sido interrumpido (sin limpiar el indicador).
isAlive	Comprueba si el hilo está vivo.

Barras de progreso

Dos hilos que avanzan a la vez – HilosConBarras.java

```
class BarraProgreso implements Runnable {
   private String nombre;

   public BarraProgreso(String nombre) {
      this.nombre = nombre;
   }

   @Override
   public void run() {
      for (int i = 1; i <= 20; i++) {
         mostrarProgreso(i);
         try {
            // pausa aleatoria entre 100 y 400 ms
            Thread.sleep((int) (Math.random() * 300) + 100);
         } catch (InterruptedException e) {
            e.printStackTrace();
         }
      }
      System.out.println("Hilo " + nombre + " ha terminado!");
   }

   private void mostrarProgreso(int paso) {
      StringBuilder barra = new StringBuilder();
      for (int j = 0; j < paso; j++) {
         barra.append("▉");
      }
      System.out.println("Hilo " + nombre + ": " + barra);
   }
}
```

```
   public static void main(String[] args) {
      Thread hilo1 = new Thread(new BarraProgreso("A"));
      Thread hilo2 = new Thread(new BarraProgreso("B"));

      hilo1.start();
      hilo2.start();

      System.out.println("Iniciando las tareas...\n");
   }
}
```

Iniciando las tareas...

Hilo A: ▉
Hilo B: ▉
Hilo A: ▉▉
Hilo B: ▉▉
Hilo B: ▉▉▉
Hilo A: ▉▉▉
...
Hilo B: ▉▉▉▉▉▉▉▉▉▉▉▉▉▉▉▉▉▉▉▉
Hilo A: ▉▉▉▉▉▉▉▉▉▉▉▉▉▉▉▉▉▉▉▉
Hilo B ha terminado!
Hilo A ha terminado!

- Las barras se **entrelazan** de forma distinta en cada ejecución: el orden de los hilos no está garantizado.
- El `main` lanza los hilos y **termina enseguida**.

Barras de progreso

¿Cómo hago que el main diga «Fin» al terminar?

Primer intento: escribir «Fin» al final del `main` (HilosConBarrasFin.java).

```
   public static void main(String[] args) {
      Thread hilo1 = new Thread(new BarraProgreso("A"));
      Thread hilo2 = new Thread(new BarraProgreso("B"));

      hilo1.start();
      hilo2.start();

      System.out.println("Iniciando las tareas...\n");
      System.out.println("Fin");   // ¡sale demasiado pronto!
   }
}
```

```
Iniciando las tareas...

Fin
Hilo B: ▉
Hilo A: ▉
Hilo A: ▉▉
Hilo B: ▉▉
Hilo A: ▉▉▉
...
```

¿Por qué sale «Fin» casi al principio?

- `start()` lanza el hilo y **vuelve enseguida**: no espera a que el hilo termine.
- Necesitamos que el `main` **espere** a los dos hilos antes de escribir «Fin»… ¿cómo?

Gestión de hilos

El método *join()*

El método *join()* permite indicar a un hilo que debe suspender su ejecución hasta que termina otro hilo de referencia.

```
public class EjemploJoin {
	public static void main(String[] args) throws InterruptedException {
		System.out.println("Inicio del programa (hilo principal)");

		Thread t1 = new Thread(new Tarea("Hilo-1"));
		Thread t2 = new Thread(new Tarea("Hilo-2"));
		Thread t3 = new Thread(new Tarea("Hilo-3"));

		t1.start();
		t2.start();
		t3.start();

		// Sincronización con join()
		System.out.println("Esperando a que terminen los hilos...");

		t1.join(); // el hilo principal espera a que termine Hilo-1
		t2.join(); // luego espera a Hilo-2
		t3.join(); // y finalmente a Hilo-3

		System.out.println("Todos los hilos han terminado.");
		System.out.println("Fin del programa (hilo principal)");
	}
}
```

```
class Tarea implements Runnable {

	private final String nombre;

	public Tarea(String nombre) {
		this.nombre = nombre;
	}

	@Override
	public void run() {
		System.out.println("Inicio de " + nombre);
		for (int i = 1; i <= 5; i++) {
			System.out.println(nombre + " ejecutando paso " + i);
			try {
				Thread.sleep(500); // simula trabajo
			} catch (InterruptedException e) {
				Thread.currentThread().interrupt();
			}
		}
		System.out.println("Fin de " + nombre);
	}
}
```

Gestión de hilos

Barras de progreso con join() – solución

- `hilo1.join()` bloquea al `main` hasta que **termina el hilo A**; después, `hilo2.join()` espera al B.
- Da igual cuál acabe antes: tras los dos `join()` ambos han terminado.
- `join()` puede lanzar `InterruptedException`: por eso el `throws` en el `main`.

```
Iniciando las tareas...

Hilo B: ▉
Hilo A: ▉
...
Hilo A ha terminado!
Hilo B: ▉▉▉
```

Gestión de hilos

El método *join()* - EJERCICIO

Modifica el ejemplo EjemploJoin (El método join()) para que:

Cada objeto TareaEncadenada debe tener:

- Un nombre para identificar el hilo.
- Una referencia al hilo anterior (por ejemplo, Thread hiloAnterior).

En el método run():

- Si existe un hilo anterior, espera a que termine antes de continuar

En la clase principal:

- Crea tres hilos (Hilo-1, Hilo-2, Hilo-3).
- El segundo debe recibir como referencia al primero, y el tercero al segundo.
- Inicia todos los hilos, pero gracias al join() dentro de cada uno, la ejecución será secuencial.

Ejercicio 1

Define una clase Corredor que **implemente la interfaz Runnable**.

- Cada corredor debe tener un nombre (por ejemplo: “Corredor 1”, “Corredor 2”…).
- En su método run(), el corredor debe avanzar en **10 pasos**.
- Tras cada paso, mostrará un mensaje como:
- Corredor 1 avanza al paso 3
- Después de cada paso, el hilo debe dormir (Thread.sleep) entre **100 y 400 ms** para simular velocidades distintas.

En el método main:

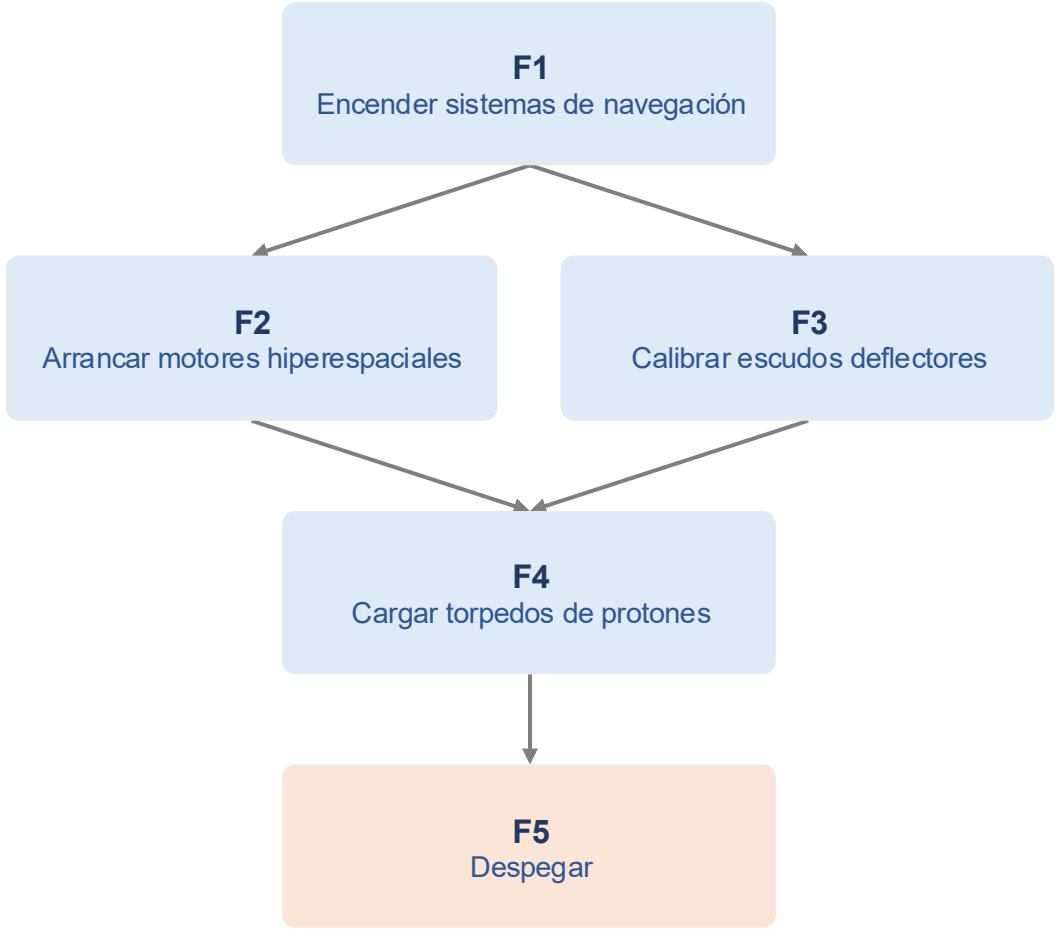
- Crea entre **3 y 5 hilos** (corredores).
- Inicia todos con start().
- Usa join() para que el programa principal espere a que terminen todos antes de imprimir:
- ¡Todos los corredores han llegado a la meta!

Gestión de hilos

join() entre hilos – Ejercicio HALCÓN MILENARIO

La Resistencia necesita poner en marcha el Halcón Milenario cuanto antes para escapar del Imperio. Varias fases se ejecutan en cadena y algunas dependen de **dos** fases anteriores: se coordinan con `join()` **entre hilos**, no solo desde el principal.

1. Implementa las fases como clases que heredan de `Thread`.
2. Cada fase espera explícitamente a sus fases anteriores.
3. Cada fase imprime «Empieza [FASE]» y «Termina [FASE]».
4. El hilo principal solo muestra «¡El Halcón Milenario despega rumbo a la galaxia!» cuando han terminado todas las fases.
5. `sleep()` solo para simular el tiempo de trabajo de cada fase.

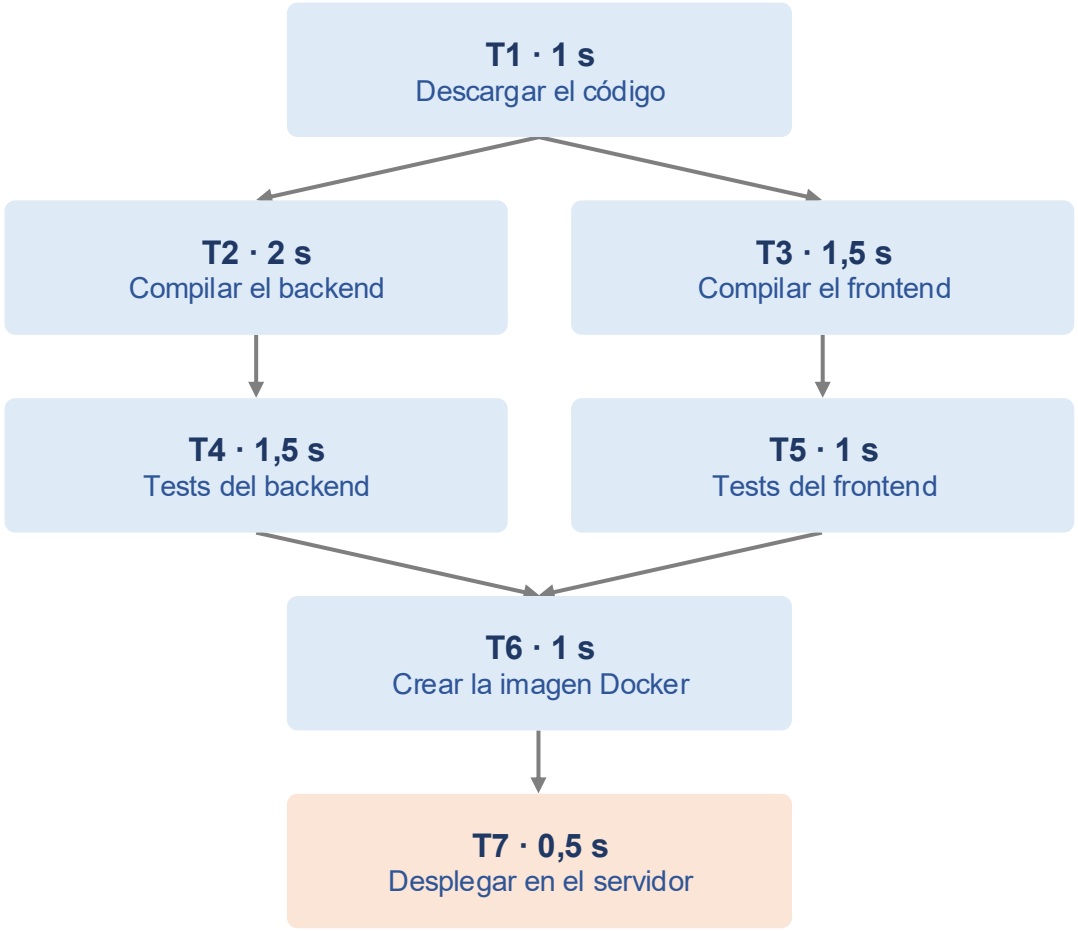


Gestión de hilos

join() entre hilos – Ejercicio DESPLIEGUE

Simula el proceso que despliega una aplicación: cada tarea es un hilo y ninguna empieza antes de que terminen las que necesita.

1. Una única clase `Tarea extends Thread` que recibe su nombre, su duración y las tareas de las que depende: `Tarea(String nombre, int ms, Thread... dependencias)`.
2. En `run()`, espera con `join()` a todas sus dependencias, trabaja con `sleep()` y muestra «Empieza» y «Termina» con los ms transcurridos desde el inicio.
3. El `main` muestra el tiempo total y lo compara con la suma de las duraciones (ejecución secuencial). ¿Qué tareas marcan el tiempo total?
4. **Ampliación:** arranca los hilos en orden inverso (T7 … T1). ¿Sigue funcionando? ¿Por qué?



Interrupciones.

Una interrupción es una petición a un hilo para que deje lo que está haciendo; el hilo decide cómo atenderla.

Se capturan con InterruptedException, que lanzan métodos como *sleep()*. Si no, el hilo lo comprueba con isInterrupted() o *interrupted()* (este limpia el indicador).

```
public class InterrupcionBasico extends Thread{

	public static void main(String[] args) throws InterruptedException {

		InterrupcionBasico t = new InterrupcionBasico();
		t.start();
		Thread.sleep(3500);
		t.interrupt(); // el main interrumpe al hilo
	}
	public void run(){
		int contador=0;
		while(true) {
			contador++;
			try {
				System.out.println(contador);
				Thread.sleep(1000);
			}catch(InterruptedException ie) {
				System.out.println("Interrupción");
				return;
			}
		}
	}
}
```

No se puede «matar» un hilo: destroy() y stop() están eliminados u obsoletos.
Se usan interrupciones para que el hilo termine de forma ordenada.

Existe destroy() para Process

Ejercicio 2

Crea un programa que lanza otro thread y se queda a la espera de que acabe antes de seguir trabajando.

Versión dos:

- Haz que el hijo se quede parado por 5 segundos y que el padre, antes de lanzar el join() lo interrumpa.
- La gestión de la interrupción será esperar dos segundos más y acabar.

Prioridad de hilos.

- El SO decide qué proceso va en ejecución.
- Dentro del proceso, se necesita un planificador para saber cuáles hilos se ejecutan (esto depende también del hardware)
- Java delega esta tarea en el planificador del SO, que suele ser apropiativo:
  - Los threads tienen una prioridad asociada.
  - Si los threads tienen misma prioridad, se asignan usando tiempo compartido
  - Si un thread con prioridad más alta pasa al estado Runnable, el planificador detiene el thread en ejecución y ejecuta el de mayor prioridad.
- *setPriority()* de la clase Thread
- Las prioridades varían en un rango de enteros comprendido entre MIN_PRIORITY y MAX_PRIORITY (entre 1 y 10; 1 prioridad mínima, 10 prioridad máxima)
- Cuando se crea un hilo, hereda la prioridad del hilo que lo creó (por defecto NORM_PRIORITY = 5) y se puede modificar en cualquier momento. La prioridad es solo una indicación: el SO puede ignorarla.

Prioridad de hilos.

```java
public class ContadorHilos extends Thread {
   String nombre;
   volatile boolean seguir = true;
   long vueltas = 0;

   public ContadorHilos(String nombre) {
      this.nombre = nombre;
   }

   @Override
   public void run() {
      // Trabajo de CPU sin sleep: compiten por el procesador
      while (seguir) {
         vueltas++;
      }
   }
}
```

```java
public class PrioridadHilos {
   public static void main(String[] args) throws InterruptedException {
      ContadorHilos hilo1 = new ContadorHilos("Hilo 1");
      hilo1.setPriority(Thread.MAX_PRIORITY);   // 10
      ContadorHilos hilo2 = new ContadorHilos("Hilo 2");
      hilo2.setPriority(Thread.MIN_PRIORITY);   // 1

      hilo1.start();
      hilo2.start();
      Thread.sleep(2000);
      hilo1.seguir = false;
      hilo2.seguir = false;
      hilo1.join();
      hilo2.join();

      System.out.println(hilo1.nombre + " (prioridad 10): "
            + hilo1.vueltas + " vueltas");
      System.out.println(hilo2.nombre + " (prioridad 1):  "
            + hilo2.vueltas + " vueltas");
   }
}
```

La prioridad es solo una indicación al sistema operativo: con núcleos libres los dos hilos avanzan casi igual. Para notar algo, lanza más hilos que núcleos (`PrioridadHilosCarga`).

Bloque 3
Compartir información

UT2. Programación multihilo

Recursos compartidos · condiciones de carrera · atomicidad · visibilidad

Compartición de información.

Podemos crear varios hilos con instancias de la clase *Thread*.

Los atributos de dichos hilos, si no son estáticos, son específicos de cada uno de ellos y por lo tanto no podrían utilizarse para compartir información.

Para que varios hilos compartan información existen varias alternativas:

- Utilizar atributos estáticos. Son comunes a todas las instancias.
- Utilizar referencias a objetos comunes accesibles desde todos los hilos.
- Utilizar atributos no estáticos de la instancia de una clase que implemente *Runnable* y construyendo los hilos a partir de dicha instancia.

Compartición de información.

- Utilizar referencias a objetos comunes accesibles desde todos los hilos.

```
public class ComparticionInfo extends Thread {

	private ObjetoComun oc;
	public static void main(String[] args) throws InterruptedException {

		ObjetoComun oc = new ObjetoComun();

		ComparticionInfo obj1 = new ComparticionInfo(oc);
		ComparticionInfo obj2 = new ComparticionInfo(oc);

		obj1.start();
		Thread.sleep(100);
		obj2.start();
	}
	public ComparticionInfo(ObjetoComun oc) {
		this.oc=oc;
	}
	public void run() {
		this.oc.variableComun++;
		System.out.println("Variable común: " + this.oc.variableComun);
	}
}


public class ObjetoComun {
		public int variableComun;
}
```

Compartición de información.

- Utilizar atributos no estáticos de la instancia de una clase que implemente Runnable y construyendo los hilos a partir de dicha instancia.

```
public class ComparticionRunnable implements Runnable {

	private int contador;

	public static void main(String[] args) throws InterruptedException{

		ComparticionRunnable cb = new ComparticionRunnable();

		new Thread(cb).start();
		Thread.sleep(100);
		new Thread(cb).start();

	}

	public void run() {
		contador++;
		System.out.println(contador);

	}

}
```

Problemas y soluciones de la programación concurrente

- Características de la programación concurrente que pueden ser fuente de errores:
  - Los recursos compartidos.
    - La problemática es diversa, ya que puede afectar a cómo se modifica el valor de una variable, se accede a los datos de una estructura de datos, se ejecuta un bloque de código o se accede a un recurso limitado.
  - El orden de ejecución.
    - La problemática tiene que ver con las dependencias que se pueden producir entre los bloques de código en función del orden de ejecución. Por ejemplo, si un bloque de código necesita como entrada datos generados como salida por otro bloque, esto provocará una dependencia, dado que en un sistema multihilo no se tiene control sobre el orden de las ejecuciones, salvo que se establezcan mecanismos de sincronización.

Problemas y soluciones de la programación concurrente

Ejemplo con recursos compartidos

```
public class VariableCompartida extends Thread {
   private static int contador;

   public static void main(String[] args) {
      Thread[] hilos = new Thread[1000];
      for (int i = 0; i < 1000; i++) {
         hilos[i] = new VariableCompartida();
         hilos[i].start();
      }
      try {
         for (Thread h : hilos) h.join();
      } catch (InterruptedException e) {
         e.printStackTrace();
      }
      System.out.println("Valor de contador: " + contador);
   }

   @Override
   public void run() {
      for (int i = 0; i < 10000; i++) {
         contador++;
      }
   }
}
```

- Las salidas obtenidas en distintas ejecuciones son distintas:
  - Valor de contador: 9472581
  - Valor de contador: 9594825
  - Valor de contador: 9570979
- Esto es debido a que el operador ++ no realiza una operación atómica (que se ejecuta sin interrupciones), sino que consta de varios pasos y la ejecución se puede interrumpir en cualquiera de ellos, provocando una condición de carrera: se pierden incrementos.
- Pasos ++:
  - Lectura valor variable.
  - Incremento en 1 del valor de la variable.
  - Escritura del valor de la variable.

Problemas y soluciones de la programación concurrente: **conceptos**

• Recursos compartidos

Un recurso compartido es un elemento del sistema que es utilizado por varios hilos de ejecución simultáneamente. Puede ser:
- Un atributo estático de una clase o un atributo no estático de un objeto compartido por todos los hilos.
- Un método estático de una clase o un método no estático de un objeto compartido por todos los hilos.
- Una conexión a una base de datos, un socket o cualquier elemento con un número limitado de instancias.

• Dependencias

No todas las tareas se pueden ejecutar en un entorno multihilo. Hay que tener la certeza de que los segmentos de código que se van a ejecutar en paralelo son independientes y el orden en el que se ejecutan es irrelevante.

Si hay dependencias, los segmentos no pueden ejecutarse en paralelo libremente.

Tres tipos de dependencias principales:
- **Dependencias de datos.** Cuando varios segmentos de código utilizan el mismo dato.
- **Dependencias de flujos.** Cuando, por existir instrucciones de control de flujo, el orden de ejecución no se puede determinar: hay dependencias potenciales que no se detectan con un análisis estático del código.
- **Dependencias de recursos.** Cuando varios segmentos de código acceden simultáneamente a recursos del procesador.

Se necesitarán mecanismos de sincronización para poder programar concurrentemente aun existiendo dependencias.

Problemas y soluciones de la programación concurrente: **conceptos**

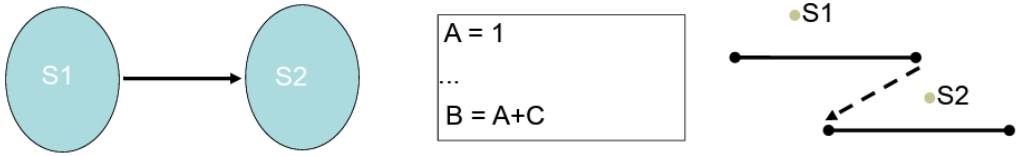
• Condiciones de Bernstein

Su cumplimiento determina si dos segmentos de código se pueden ejecutar en paralelo.

Dos segmentos de código S1 y S2 son independientes si:

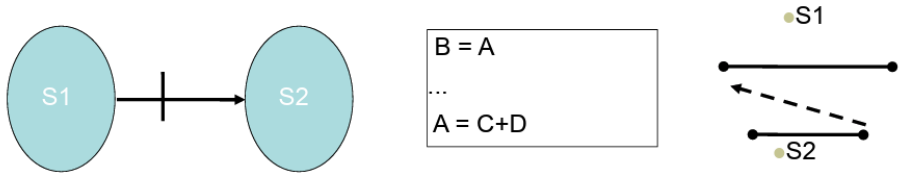
- Las entradas de S2 son distintas de las salidas de S1.

En caso contrario, habría **dependencia de flujo.**



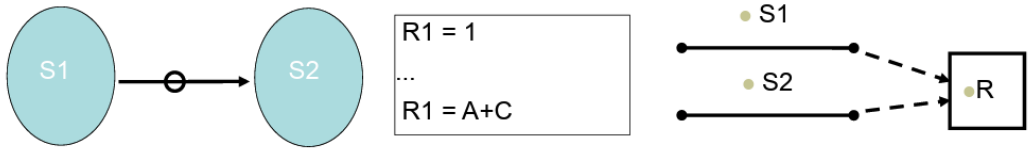
- Las entradas de S1 son distintas de las salidas de S2.

En caso contrario, se produciría **antidependencia.**



- Las salidas de S1 son distintas de las salidas de S2.

En caso contrario, habría **dependencia de salida**.



Problemas y soluciones de la programación concurrente: **conceptos**

- **Operación y acceso atómicos**

Una acción atómica es aquella que se ejecuta sin interrupciones, de una única vez.

Otro hilo o proceso no puede leer o modificar datos relacionados mientras se esté realizando la operación.

Una instrucción máquina suele ser atómica, pero una operación como x++ se compone de varias (leer, sumar, escribir) y no lo es.

La palabra reservada *volatile* NO garantiza atomicidad: garantiza visibilidad.

Cada lectura ve el último valor escrito por cualquier hilo **(no se queda en caché o registros).** Pero x++ sobre una variable volatile sigue sin ser atómico.

Para operaciones atómicas: clases de java.util.concurrent.atomic (AtomicInteger…) o una sección crítica.

```
public class EjemploVolatile extends Thread {

	public static volatile boolean stop = false;

	public static void main(String args[]) throws InterruptedException {
		Thread testThread = new EjemploVolatile();
		testThread.start();
		Thread.sleep(5000);

		stop = true;

		System.out.println("MAIN - stop vale: " + stop);

		testThread.join();
	}

	public void run() {
		int i = 1;
			while (!stop) {
			i++;
		}
		System.out.println("THREAD - stop i=" + i);
	}
}
```

Problemas y soluciones de la programación concurrente: **conceptos**

- **Sección crítica**

Dentro de un programa multihilo, es el bloque de código que accede a recursos compartidos, por lo que debe ser accedido por un único hilo de ejecución.

Cuando un hilo está ejecutando su sección crítica, ningún otro hilo puede ejecutar su correspondiente sección crítica.

- Si otro hilo quiere ejecutar su sección crítica, se bloqueará hasta que el primer hilo haya acabado su sección crítica.
- Se establece una relación de “antes-después” en la ejecución de sección crítica.

Hay que determinar un conjunto de reglas que permitan a los hilos cooperar.

Garantizar que a la sección crítica sólo acceda un hilo de ejecución es lo que se conoce como **exclusión mutua**.

El código de la sección crítica debe terminar en un tiempo determinado y hay que implementar mecanismos que eviten la inanición

```
public class ExclusionMutua {

	static long cuenta = 0; /* variable compartida */

	private static class Incrementa extends Thread {
		public void run() {
			for (int i = 0; i < 1000000; i++) {
				cuenta++; /* región crítica */
			}
		}
	}

	public static void main(String[] args) throws InterruptedException {
		Thread h1 = new Incrementa(); /* hilo 1 */
		Thread h2 = new Incrementa(); /* hilo 2 */

		h1.start();
		h2.start();

		h1.join();
		h2.join();

		System.out.println("contador = " + cuenta);
	}
}
```

Problemas y soluciones de la programación concurrente: **conceptos**

- **Ejemplo. Cuando dos hilos acceden al mismo objeto**

```
public class Contador {
	private long cuenta = 0; /* estado */

	public Contador(long valorInicial) {
		cuenta = valorInicial;
	}

	public void incrementar() {
		cuenta++; /* modifica el estado */
	}

	public long valor() { /* devuelve el valor */
		return cuenta;
	}
}
```

```
public class PruebaContador {
   private static class Incrementa extends Thread {
      Contador contador;

      public Incrementa(Contador c) {
         contador = c;
      }

      @Override
      public void run() {
         for (int i = 0; i < 100_000; i++) {
            contador.incrementar(); /* región crítica */
         }
      }
   }

   public static void main(String[] args) throws InterruptedException {
      Contador contador = new Contador(0);
      Thread hilo1 = new Incrementa(contador);
      Thread hilo2 = new Incrementa(contador);
      /* las dos hebras comparten el objeto contador */
      hilo1.start();
      hilo2.start();
      hilo1.join();
      hilo2.join();
      System.out.println("Esperado: 200000  Obtenido: " + contador.valor());
   }
}
```

Problemas de la programación concurrente

Condiciones de carrera

Cuando dos bloques de código concurrente tienen dependencias entre los datos de entrada y salida y no se ejecutan en el orden correcto (dependencia de flujo o antidependencia).

```
public class EjemploCondicionCarrera {
	public static void main(String[] args) {
		Contador contadorCompartido = new Contador();

		// Creamos dos hilos que incrementan el mismo contador
		Thread hilo1 = new Thread(new TareaIncremento(contadorCompartido));
		Thread hilo2 = new Thread(new TareaIncremento(contadorCompartido));

		hilo1.start();
		hilo2.start();

		try {
			hilo1.join();
			hilo2.join();
		} catch (InterruptedException e) {
			e.printStackTrace();
		}

		System.out.println("Valor final del contador: " +
		contadorCompartido.getValor());
	}
}
```

```
//Clase que contiene una variable compartida entre varios hilos.
class Contador {
	private int valor = 0;

	public void incrementar() {
	// No está sincronizado → posible condición de carrera
		valor = valor + 1;
	}

	public int getValor() {
		return valor;
	}
}

// Clase que representa una tarea que incrementa muchas veces el contador.
class TareaIncremento implements Runnable {
	private Contador contador;

	public TareaIncremento(Contador contador) {
		this.contador = contador;
	}

	@Override
	public void run() {
		for (int i = 0; i < 100000; i++) {
		contador.incrementar();
		}
	}
}
```

Esto genera una condición de carrera, porque el resultado depende del orden no controlado de ejecución de los hilos.

Problemas de la programación concurrente

Inconsistencia de memoria

Este fenómeno ocurre cuando un hilo ve un estado desactualizado o incompleto de una variable compartida, porque los cambios hechos por otro hilo no se han propagado aún a la memoria principal.

```
public class EjemploIM extends Thread {
   // Sin volatile: el hilo puede no ver el cambio
   static boolean listo = false;

   public static void main(String[] args) throws InterruptedException {
      new EjemploIM().start();
      Thread.sleep(1000);
      listo = true;   // el main cambia la variable
      System.out.println("main: listo = true");
   }

   public void run() {
      while (!listo) { }   // puede no terminar nunca
      System.out.println("El hilo ha visto el cambio");
   }
}

// Solución: static volatile boolean listo
```

Bloque 4
Sincronización

UT2. Programación multihilo

synchronized · métodos y bloques · variables atómicas

Métodos de sincronización.

Sincronización mediante monitores - *synchronized*

La sincronización es posible gracias al uso de monitores, un mecanismo que permite controlar el acceso concurrente a objetos en conflicto.

Los monitores tienen la capacidad de bloquear un objeto mientras un hilo está accediendo a él, de modo que no permite el acceso por parte de otros hilos. Esto se conoce como **exclusión mutua**.

Cuando el hilo que bloqueaba el objeto termina sus operaciones, desbloquea el objeto para que puedan acceder otros hilos. De este modo podemos sincronizar el acceso a un objeto compartido.

Todos los objetos tienen asociado un monitor, por lo que pueden sincronizarse. Para acceder a este monitor se utiliza la palabra reservada **`synchronized`**, que podemos emplear de dos formas:

- Para declarar métodos como **`synchronized`** en el objeto en conflicto, de modo que se entra en el monitor cuando se acceda a él, asegurando así una exclusión mutua.
    - **public synchronized void incrementar()**
- Para declarar solamente una sección de código como **`synchronized`**, de modo que entramos en el monitor sobre el objeto compartido dentro de dicha sección, asegurando así el acceso exclusivo a este.

Esta segunda opción es la más recomendable, ya que es preferible sincronizar la mínima parte de código que sea posible.

Métodos de sincronización.

Sincronización mediante monitores – *synchronized* – EJERCICIO CONTAR

Crea un programa que genere N threads.

Cada thread, por 1000 veces, ejecuta inc1() y inc2()

Inc1() incrementa una variable global c1

Inc2() incrementa otra variable global c2

Al final se indicará el valor final de c1 y c2

Métodos de sincronización.

Sincronización mediante monitores – *synchronized* – EJERCICIO VOCALES

Crea un programa que lanza 5 hilos que trabajan sobre un determinado texto.

Cada hilo se encarga de contar una vocal distinta, pero todos los hilos actualizan la misma variable.

El objetivo es indicar cuantas vocales hay en el texto.

Métodos de sincronización.

Sincronización mediante monitores – *synchronized* – sentencias sincronizadas

Los métodos sincronizados usan un monitor que afecta a todo el objeto, bloqueando todos los otros métodos sincronizados.

Esto puede causar que algunas operaciones que sí se podrían paralelizar, en realidad no se puedan hacer.

Sincronizar sentencias permite más versatilidad y una sincronización menos gruesa

Se puede usar synchronized para sincronizar una región de código determinada.

Además, se puede especificar el objeto que proporciona el monitor en vez de usar al objeto por defecto que se está ejecutando.

- Esto permite crear distintos objetos que hagan de monitores y bloqueen los hilos de forma independiente

Métodos de sincronización.

Sincronización mediante monitores – *synchronized* – sentencias sincronizadas

Cualquier objeto puede ser usado para ser un monitor.

La clase Object tiene las características para ser un monitor, y como todas las clases heredan de Object…

Se puede usar directamente la clase Object

Para sincronizar sentencias, se tiene que usar el mismo objeto (si uso monitores distintos, las sentencias se ejecutarán en paralelo).

```
// atributo, no variable local:

private Object mutex = new Object();


public void miMetodo(){

    //[…]

    synchronized(mutex) {

            /* sentencias sincronizadas usando

            el objeto mutex como monitor */

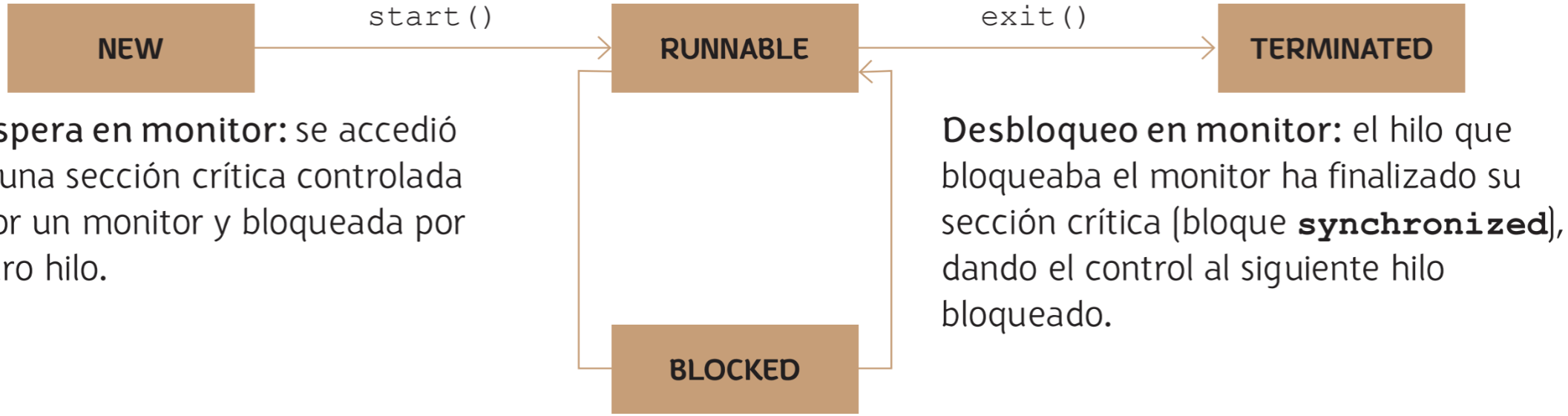
    }

    //[…]

} //acaba el método
```

Métodos de sincronización.

Sincronización mediante monitores – *synchronized* – sentencias sincronizadas



Espera en monitor: se accedió a una sección crítica controlada por un monitor y bloqueada por otro hilo.

Desbloqueo en monitor: el hilo que bloqueaba el monitor ha finalizado su sección crítica (bloque **`synchronized`**), dando el control al siguiente hilo bloqueado.

Métodos de sincronización.

Sincronización mediante monitores – *synchronized* – sentencias sincronizadas

Objetos para sincronizar

Cualquier objeto puede ser usado para ser un monitor.

La clase Object tiene las características para ser un monitor, y como todas las clases heredan de Object…

Se puede usar directamente la clase Object

Para sincronizar sentencias, se tiene que usar el mismo objeto (si uso monitores distintos, las sentencias se ejecutarán en paralelo).

Sincronización reentrante

Si un hilo ya tiene el monitor porque ha accedido a una sección crítica, podrá llamar desde allí dentro otro método o sección crítica que use el mismo objeto como monitor (ya que ya tiene el control sobre el cerrojo…)

Métodos de sincronización.

Sincronización mediante monitores – *synchronized* – sentencias sincronizadas

```
public class ContadorSegmento {

        private int valor = 0;
        private final Object lock = new Object(); // monitor explícito

        public ContadorSegmento(int valorInicial) {
        valor=valorInicial;
        }

        public void incrementar() {
        // Solo esta parte está protegida
                synchronized (lock) {
                                valor++;
                }
        // El resto del método puede ejecutarse en paralelo
        }

        public int getValor() {
                synchronized (lock) { return valor; }
        }
}
```

Métodos de sincronización.

Sincronización mediante monitores – synchronized – Ejercicio CONTAR 2

Recupera el ejercicio donde sumabas dos variables c1 y c2 y modifícalo para que funcione de forma más eficiente usando la sincronización de sentencias

Métodos de sincronización.

Sincronización mediante monitores – *synchronized* – Diferencias

Método sincronizado (synchronized en la declaración)

Cuando declaras un método como `synchronized`, **todo el método queda protegido** por el monitor del objeto o de la clase (si es estático).

Solo **un hilo puede ejecutar ese método a la vez** sobre la misma instancia.

Comportamiento:

- Cuando un hilo entra en `incrementar()`, **bloquea el monitor del objeto `Contador`**.
- Ningún otro hilo puede ejecutar **ningún otro método sincronizado** del mismo objeto hasta que el primero termine.
- Es una **sincronización total** sobre el objeto.

```
public class Contador {
        private int valor = 0;

        // Método sincronizado
        public synchronized void incrementar() {
                valor++;
        }

        public synchronized int getValor() {
                return valor;
        }
}
```

Ventaja:	simple y segura.
Inconveniente:	**bloquea más de lo necesario** si el método realiza tareas que no afectan a datos compartidos.

Métodos de sincronización.

Sincronización mediante monitores – *synchronized* – Diferencias

Bloque sincronizado (`synchronized` dentro del método)

Un bloque sincronizado permite **delimitar solo una parte del código** que necesita acceso exclusivo.

De esta forma, **solo se sincroniza la sección crítica**, no todo el método.

Comportamiento:

- El hilo bloquea **solo el objeto lock** durante el bloque sincronizado.
- Otros hilos pueden ejecutar otras partes del código sin problema.
- Es más **eficiente** cuando el método tiene partes no críticas.

```
public class Contador {
        private int valor = 0;
        private final Object lock = new Object(); // monitor explícito

        public void incrementar() {
        // Solo esta parte está protegida
                synchronized (lock) {
                                valor++;
                }
        // El resto del método puede ejecutarse en paralelo
        }

        public int getValor() {
                synchronized (lock) { return valor; }
        }
}
```

Ventaja:	**mayor rendimiento**, al reducir la zona crítica.
Inconveniente:	más complejo de mantener y requiere elegir bien el objeto monitor.

Métodos de sincronización.

Sincronización mediante monitores – *synchronized* – Diferencias

Aspecto	Método `synchronized`	Bloque `synchronized`
Alcance del bloqueo	Todo el método	Solo la sección definida
Monitor usado	El propio objeto (`this`) o la clase si es estático	El objeto especificado en el paréntesis
Facilidad de uso	Muy sencillo	Requiere definir el bloque y el objeto monitor
Eficiencia	Menor, puede bloquear código no crítico	Mayor, solo bloquea lo necesario
Uso típico	Métodos cortos y críticos	Métodos largos o con partes no compartidas

Métodos de sincronización.

Sincronización mediante monitores – synchronized – Ejercicio COMPARACIÓN

- Implementa una clase que contenga una variable compartida y un método incrementador declarado como `synchronized`.

  Crea varios hilos, N, que invoquen repetidamente ese método (por ejemplo, i=500) y mide el tiempo que tarda la ejecución completa.

- Implementa una segunda versión donde el método incrementador **NO** sea `synchronized`, pero contenga un **bloque `synchronized`** que proteja únicamente la operación crítica sobre la variable compartida.

  Vuelve a lanzar los hilos y mide el tiempo de ejecución.

- Compara los resultados e indica cuál de las dos implementaciones es más eficiente y por qué.

Problemas y soluciones de la programación concurrente: **conceptos**

- **Synchronized**

`synchronized`

Palabra clave para garantizar **exclusión mutua** entre hilos.

Su uso típico es proteger variables compartidas, contadores, colecciones, etc.

Riesgo	Descripción
🚫 **Bloqueo mutuo (Deadlock)**	Si dos hilos se esperan mutuamente, el programa puede quedarse colgado.
Pérdida de rendimiento	Sincronizar en exceso puede ralentizar la aplicación.
Granularidad	Mejor sincronizar solo las secciones críticas, no métodos enteros si no es necesario.

Métodos de sincronización.

Variables atómicas – java.util.concurrent.atomic

- Clases cuyas operaciones son **atómicas sin usar synchronized**: `AtomicInteger`, `AtomicLong`, `AtomicBoolean`, `AtomicReference`…
- Usan una instrucción del procesador de tipo **compare-and-set (CAS)**: más rápida que un cerrojo cuando solo hay que proteger una variable.
- Solo protegen **una operación sobre una variable**. Si hay que modificar varias variables juntas, sigue haciendo falta una sección crítica.

Método de AtomicInteger	Qué hace
`get()` / `set(v)`	Lee / escribe el valor (con visibilidad garantizada, como volatile)
`incrementAndGet()`	Suma 1 y devuelve el valor nuevo (equivale a ++x de forma atómica)
`getAndIncrement()`	Devuelve el valor actual y después suma 1 (x++ atómico)
`addAndGet(n)`	Suma n y devuelve el valor nuevo
`compareAndSet(esp, nuevo)`	Si el valor es esp, lo cambia por nuevo y devuelve true; si no, devuelve false

Métodos de sincronización.

Variables atómicas – ejemplo

El mismo programa que daba resultados distintos en cada ejecución (VariableCompartida), ahora con un AtomicInteger.

- `contador.incrementAndGet()` es atómico: ningún incremento se pierde.
- El resultado es **siempre 1.000.000**.
- No hace falta synchronized ni ningún objeto monitor.

```
import java.util.concurrent.atomic.AtomicInteger;

public class ContadorAtomico extends Thread {
   static AtomicInteger contador = new AtomicInteger(0);

   public void run() {
      for (int i = 0; i < 1000; i++) {
         contador.incrementAndGet();   // atómico
      }
   }

   public static void main(String[] args) throws InterruptedException {
      Thread[] hilos = new Thread[1000];
      for (int i = 0; i < 1000; i++) {
         hilos[i] = new ContadorAtomico();
         hilos[i].start();
      }
      for (Thread h : hilos) h.join();
      System.out.println("Valor: " + contador.get());
   }
}
```

Continuará…